

Chapter 10: Database Performance Tuning

একটি Database System যখন ছোট থাকে, তখন Query Performance নিয়ে খুব বেশি সমস্যা দেখা যায় না। কিন্তু Database-এর Data বাড়তে থাকে, User সংখ্যা বৃদ্ধি পায় এবং একই সময়ে অনেক Transaction ও Query চলতে শুরু করলে Database ধীর হয়ে যেতে পারে।

একটি Query আগে যেখানে ১ সেকেন্ডে Result দিত, Data কয়েক কোটি Row হয়ে যাওয়ার পর সেটি কয়েক মিনিটও নিতে পারে।

তখন প্রশ্ন আসে—

Database ধীর কেন হচ্ছে?

এই প্রশ্নের উত্তর খুঁজে বের করা এবং Database-এর Performance উন্নত করার প্রক্রিয়াই হলো **Database Performance Tuning**।

Performance Tuning-এর মাধ্যমে আমরা সাধারণত—

- Query দ্রুত করি
- Unnecessary I/O কমাই
- CPU Usage নিয়ন্ত্রণ করি
- Memory Usage Optimize করি
- সঠিক Index ব্যবহার করি
- Execution Plan বিশ্লেষণ করি
- Wait Events বিশ্লেষণ করি
- Database Workload পর্যবেক্ষণ করি

১. SQL Tuning

SQL Tuning কী?

SQL Tuning হলো SQL Query-কে এমনভাবে পরিবর্তন ও Optimize করার প্রক্রিয়া, যাতে Query কম Resource ব্যবহার করে দ্রুত Result প্রদান করতে পারে।

ধরুন একটি Query—

```
SELECT *
```

```
FROM ORDERS
```

```
WHERE CUSTOMER_ID = 1001;
```

ORDERS Table-এ যদি কয়েক কোটি Row থাকে এবং CUSTOMER_ID-এর ওপর উপযুক্ত Index না থাকে, তাহলে Database-কে অনেক বেশি Data Scan করতে হতে পারে।

SQL Tuning-এর সময় আমরা দেখতে পারি—

- Query কীভাবে Data Access করছে?
- কোন Table বড়?
- কোন Column-এ Filter করা হচ্ছে?
- Index আছে কি?
- Unnecessary Column নেওয়া হচ্ছে কি?
- Join সঠিকভাবে করা হয়েছে কি?
- Function বা Expression Query-কে ধীর করছে কি?

SELECT * কেন এড়ানো উচিত?

প্রয়োজন না থাকলে—

SELECT *

FROM CUSTOMERS;

এর পরিবর্তে নির্দিষ্ট Column নির্বাচন করা ভালো।

SELECT CUSTOMER_ID, CUSTOMER_NAME, PHONE

FROM CUSTOMERS;

এতে অপ্রয়োজনীয় Data Read ও Transfer কমানো যেতে পারে।

২. Execution Plan

Execution Plan কী?

Execution Plan হলো Database Engine কীভাবে একটি SQL Query Execute করবে তার পরিকল্পনা।

একটি Query চালানোর আগে Database Optimizer বিভিন্ন সম্ভাব্য Execution Method বিবেচনা করতে পারে।

যেমন—

- Table Scan

- Index Scan
- Index Seek
- Nested Loop
- Hash Join
- Merge Join
- Sort
- Aggregate

এরপর Optimizer Cost এবং অন্যান্য তথ্যের ভিত্তিতে একটি Execution Plan নির্বাচন করে।

উদাহরণ

ধরুন—

```
SELECT *  
FROM CUSTOMERS  
WHERE CUSTOMER_ID = 1001;
```

যদি CUSTOMER_ID-এর ওপর উপযুক্ত Index থাকে, Database হয়তো Index ব্যবহার করে দ্রুত নির্দিষ্ট Row খুঁজে বের করতে পারে।

অন্যদিকে Index না থাকলে পুরো Table Scan করতে হতে পারে।

```
Query  
↓  
Optimizer  
↓  
Execution Plan  
↓  
Data Access  
↓  
Result
```

Performance Tuning-এর সময় Execution Plan দেখা অত্যন্ত গুরুত্বপূর্ণ।

৩. Explain Plan

Explain Plan কী?

Explain Plan হলো Query কীভাবে Execute হতে পারে তার পরিকল্পনা দেখার একটি পদ্ধতি।

বিশেষ করে Oracle Database-এ EXPLAIN PLAN ব্যবহার করে Query-এর সম্ভাব্য Execution Plan দেখা যায়।

উদাহরণ:

```
EXPLAIN PLAN FOR  
  
SELECT *  
  
FROM CUSTOMERS  
  
WHERE CUSTOMER_ID = 1001;
```

তারপর Plan দেখার জন্য Oracle-এর Plan Table ব্যবহার করা যায়।

```
SELECT *  
  
FROM TABLE(DBMS_XPLAN.DISPLAY);
```

এখান থেকে বোঝা যায় Database কোন Access Path ব্যবহার করার পরিকল্পনা করেছে।

গুরুত্বপূর্ণ পার্থক্য

Explain Plan সাধারণত Optimizer-এর তৈরি সম্ভাব্য Plan দেখায়।

অন্যদিকে **Actual Execution Plan** Query বাস্তবে চালানোর পরে কীভাবে Execute হয়েছে সে সম্পর্কে আরও বাস্তব তথ্য দিতে পারে।

8. Index Tuning

Index Tuning কী?

Index হলো Database-এর এমন একটি Data Structure, যা নির্দিষ্ট Data দ্রুত খুঁজে পেতে সাহায্য করতে পারে।

ধরুন একটি Table-এ ১০ কোটি Customer Record রয়েছে।

Query:

```
SELECT *  
  
FROM CUSTOMERS  
  
WHERE CUSTOMER_CODE = 'C10001';
```

CUSTOMER_CODE-এর ওপর উপযুক্ত Index থাকলে Database অনেক কম Data Access করে কাঙ্ক্ষিত Row খুঁজে পেতে পারে।

Index কেন গুরুত্বপূর্ণ?

Index সঠিকভাবে ব্যবহার করলে—

- Search দ্রুত হতে পারে
- Table Scan কমতে পারে
- I/O কমতে পারে
- Query Response Time কমতে পারে

তবে বেশি Index করাও ভালো নয়।

কারণ INSERT, UPDATE এবং DELETE করার সময় Index Maintain করতে হয়।

অতএব—

Index Tuning-এর লক্ষ্য শুধু Index তৈরি করা নয়; সঠিক Column-এ সঠিক Index তৈরি করা।

৫. Statistics

Statistics কী?

Database Optimizer Query-এর জন্য ভালো Execution Plan নির্বাচন করতে বিভিন্ন তথ্য ব্যবহার করে। এর মধ্যে Statistics অত্যন্ত গুরুত্বপূর্ণ।

Statistics থেকে Database সম্পর্কে বিভিন্ন তথ্য পাওয়া যায়, যেমন—

- Data Distribution
- Number of Rows
- Column-এর Data Distribution
- Distinct Values
- Data Selectivity

ধরুন একটি Column-এ অধিকাংশ Row-এর Value একই, কিন্তু কিছু Value খুব Rare।

Optimizer যদি Data Distribution সম্পর্কে সঠিক Statistics পায়, তাহলে Query-এর জন্য আরও উপযুক্ত Plan নির্বাচন করতে পারে।

Statistics পুরোনো হলে কী হতে পারে?

Data অনেক পরিবর্তন হলেও Statistics Update না হলে Optimizer ভুল Cardinality Estimate করতে পারে।

এর ফলে—

Wrong Statistics



Wrong Estimate



Poor Execution Plan



Slow Query

তাই Database Environment অনুযায়ী Statistics নিয়মিত Monitor ও Update করা গুরুত্বপূর্ণ।

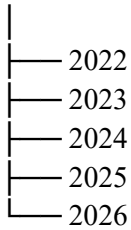
৬. Partitioning

Partitioning কী?

Partitioning হলো একটি বড় Table বা Index-কে ছোট ছোট Logical অংশে ভাগ করার পদ্ধতি।

ধরুন একটি SALES Table-এ ১০ বছরের Data রয়েছে।

SALES



Date অনুযায়ী Partition করলে নির্দিষ্ট বছরের Data নিয়ে Query করার সময় Database অপয়োজনীয় Partition Access না করে প্রয়োজনীয় Partition-এ সীমাবদ্ধ থাকতে পারে।

এটিকে **Partition Pruning** বলা হয়।

Partitioning-এর সুবিধা

- বড় Table Management সহজ হতে পারে
- Query Performance উন্নত হতে পারে
- Maintenance সহজ হতে পারে
- পুরোনো Data আলাদাভাবে Manage করা যায়
- বড় Data Set পরিচালনা করা সহজ হয়

Partitioning সাধারণত বড় Table এবং নির্দিষ্ট Access Pattern-এর ক্ষেত্রে বেশি কার্যকর।

৭. Parallel Query

Parallel Query কী?

একটি Query-এর কাজ একাধিক Processing Unit বা Parallel Worker-এর মাধ্যমে একই সময়ে সম্পন্ন করার পদ্ধতিকে **Parallel Query Processing** বলা হয়।

সাধারণভাবে—

Serial Query

Query

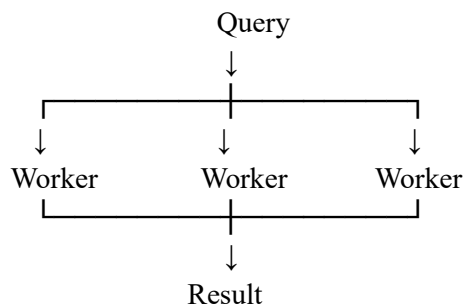


Worker



Result

Parallel Processing-এ—



বড় Data Processing, Data Warehouse এবং Analytical Workload-এর ক্ষেত্রে Parallel Query কার্যকর হতে পারে।

তবে সব Query-তে Parallelism ভালো নয়।

অতিরিক্ত Parallelism হলে—

- CPU Usage বাড়তে পারে
- Memory Consumption বাড়তে পারে
- অন্য Query-এর Performance কমতে পারে

তাই Workload অনুযায়ী Parallelism Tune করতে হয়।

৮. Memory Tuning

Memory Tuning কী?

Database Server-এর Memory সঠিকভাবে ব্যবহার নিশ্চিত করার প্রক্রিয়াকে **Memory Tuning** বলা হয়।

Database সাধারণত Memory ব্যবহার করে—

- Data Cache
- Query Processing
- Execution Plan
- Sorting
- Hash Operation
- Connection Management

এর জন্য।

যদি Database পর্যাপ্ত Memory না পায়, তাহলে Disk I/O বৃদ্ধি পেতে পারে এবং Query Performance কমে যেতে পারে।

Memory Tuning-এর লক্ষ্য

Available Memory

↓

Database Memory

↓

Efficient Cache

↓

Less Disk I/O

↓

Better Performance

তবে Database-কে পুরো Server Memory দিয়ে দেওয়াও সঠিক নয়। Operating System এবং অন্যান্য প্রয়োজনীয় Service-এর জন্যও পর্যাপ্ত Memory রাখতে হয়।

৯. Wait Events

Wait Event কী?

Database কোনো কাজ সম্পন্ন করার জন্য যখন কোনো Resource-এর জন্য অপেক্ষা করে, তখন সেই অপেক্ষাকে **Wait Event** হিসেবে পর্যবেক্ষণ করা যায়।

উদাহরণস্বরূপ Database অপেক্ষা করতে পারে—

- Disk I/O
- Lock
- Network
- CPU Scheduling
- Memory-related Resource
- অন্য Transaction

এর অর্থ হলো—

Database ধীর মানেই CPU ধীর—এমন নয়। Database আসলে কীসের জন্য Wait করছে তা জানা গুরুত্বপূর্ণ।

Wait Event Analysis

ধরুন একটি Query ধীর।

আমরা দেখলাম Database দীর্ঘ সময় ধরে Disk I/O-এর জন্য অপেক্ষা করছে।

তাহলে সম্ভাব্য সমস্যা হতে পারে—

Slow Query



High I/O Wait



Storage / Query Access সমস্যা



Execution Plan / Index / Storage বিশ্লেষণ

অন্যদিকে যদি Lock-related Wait বেশি হয়, তাহলে Concurrent Transaction এবং Blocking বিশ্লেষণ করতে হবে।

১০. AWR Report

AWR কী?

AWR (Automatic Workload Repository) হলো Oracle Database-এর একটি গুরুত্বপূর্ণ Performance Monitoring এবং Diagnostic Feature।

AWR Database Workload-এর বিভিন্ন গুরুত্বপূর্ণ Performance Information সংরক্ষণ করে।

AWR Report ব্যবহার করে DBA বিভিন্ন সময়ের Database Performance বিশ্লেষণ করতে পারেন।

AWR Report-এ কী দেখা যায়?

সাধারণভাবে দেখা যেতে পারে—

- Database Load
- CPU Usage
- I/O Activity
- Top SQL
- Wait Events
- Load Profile
- Instance Activity
- SQL Statistics
- Database Time

উদাহরণ:

Performance Problem

↓

AWR Report

↓

Top SQL

↓

Wait Events

↓

Resource Usage

↓

Root Cause Analysis

AWR বিশেষ করে Oracle Database Performance Troubleshooting-এর ক্ষেত্রে অত্যন্ত গুরুত্বপূর্ণ।

১১. ASH Report

ASH কী?

ASH (Active Session History) Oracle Database-এর Active Session Activity পর্যবেক্ষণের জন্য ব্যবহৃত একটি গুরুত্বপূর্ণ Performance Diagnostic Feature।

ASH-এর মাধ্যমে নির্দিষ্ট সময়ে Active Session-গুলো কী করছিল, কোন SQL চালাচ্ছিল এবং কীসের জন্য Wait করছিল—এসব বিশ্লেষণ করা যায়।

ASH Report কেন ব্যবহার করবেন?

ধরুন সকাল ১০টা থেকে ১০টা ১৫ মিনিটের মধ্যে Database খুব Slow ছিল।

কিন্তু এখন Database স্বাভাবিক।

এই অবস্থায় Historical Activity বিশ্লেষণ করতে ASH খুব কার্যকর হতে পারে।

Problem Time



ASH Data



Active Sessions



SQL + Wait Events



Problem Identification

AWR বনাম ASH

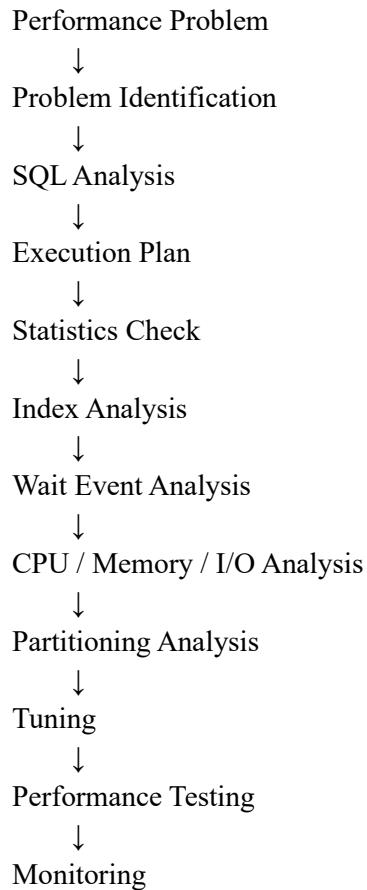
বিষয়	AWR	ASH
পূর্ণরূপ	Automatic Workload Repository	Active Session History
মূল উদ্দেশ্য	Overall Workload Analysis	Active Session Analysis
Scope	তুলনামূলকভাবে Broad	বেশি Session-focused
Top SQL	✓	✓
Wait Analysis	✓	✓
Historical Analysis	✓	✓
নির্দিষ্ট সময়ের Active Session	সীমিত	বিশেষভাবে উপযোগী

Database Performance Tuning-এর সম্পূর্ণ Workflow

একটি Database Slow হলে সরাসরি Index তৈরি করা উচিত নয়।

প্রথমে সমস্যাটি কোথায় তা খুঁজে বের করতে হবে।

একটি ভালো Performance Tuning Process হতে পারে—



একটি বাস্তব উদাহরণ

ধরুন একটি E-Commerce Database-এ নিচের Query-টি ধীরে চলছে—

```
SELECT  
  
    ORDER_ID,  
  
    CUSTOMER_ID,  
  
    ORDER_DATE,  
  
    TOTAL_AMOUNT  
FROM ORDERS  
  
WHERE CUSTOMER_ID = 1001;
```

প্রথমে Query-এর Execution Plan দেখা হলো।

দেখা গেল Database পুরো Table Scan করছে।

তারপর দেখা হলো CUSTOMER_ID-এর ওপর উপযুক্ত Index নেই।

প্রয়োজনীয় Workload Analysis-এর পরে যদি Index উপযুক্ত বলে প্রমাণিত হয়, তাহলে Index তৈরি করা যেতে পারে।

```
CREATE INDEX IX_ORDERS_CUSTOMER_ID  
ON ORDERS(CUSTOMER_ID);
```

এরপর Query আবার Test করতে হবে।

Before Tuning

↓
Table Scan
↓
High I/O
↓
Slow Query

After Tuning

↓
Appropriate Index Access
↓
Less I/O
↓
Faster Query

তবে বাস্তব Production Environment-এ Index তৈরি করার আগে Write Overhead, Storage Cost, Existing Index, Query Pattern এবং Execution Plan যাচাই করা উচিত।

Performance Tuning-এর গুরুত্বপূর্ণ নীতি

১. আগে সমস্যা বুঝুন

অনুমান করে Tuning করবেন না।

২. Execution Plan দেখুন

Database Query কীভাবে Execute করছে তা বুঝুন।

৩. Wait Event বিশ্লেষণ করুন

Database আসলে কোথায় সময় ব্যয় করছে তা খুঁজে বের করুন।

৪. Statistics পরীক্ষা করুন

Optimizer-এর কাছে Data সম্পর্কে সঠিক তথ্য আছে কি না দেখুন।

৫. Index অন্ধভাবে তৈরি করবেন না

প্রতিটি Index-এর Performance Benefit এবং Maintenance Cost বিবেচনা করুন।

৬. Hardware এবং Query—দুটিই দেখুন

সব Performance Problem Query-এর কারণে হয় না। CPU, RAM, Storage এবং Network-ও সমস্যা তৈরি করতে পারে।

৭. Tuning-এর পরে আবার Measure করুন

Tuning সফল হয়েছে কিনা তা Measurement-এর মাধ্যমে নিশ্চিত করতে হবে।

অধ্যায়ের সারসংক্ষেপ

Database Performance Tuning হলো Database-এর Performance বিশ্লেষণ, সমস্যা শনাক্তকরণ এবং উপযুক্ত পরিবর্তনের মাধ্যমে Database-কে আরও দ্রুত ও কার্যকর করার প্রক্রিয়া।

এই অধ্যায়ে আমরা দেখলাম—

- **SQL Tuning** Query Optimize করতে সাহায্য করে।
- **Execution Plan** Database কীভাবে Query Execute করছে তা বুঝতে সাহায্য করে।
- **Explain Plan** Query-এর সম্ভাব্য Execution Strategy বিশ্লেষণে ব্যবহৃত হয়।
- **Index Tuning** Data Access দ্রুত করতে সাহায্য করতে পারে।
- **Statistics** Optimizer-কে সঠিক Plan নির্বাচন করতে সাহায্য করে।
- **Partitioning** বড় Table Management ও কিছু Query-এর Performance উন্নত করতে পারে।
- **Parallel Query** বড় Workload-কে একাধিক Worker-এর মাধ্যমে Process করতে পারে।
- **Memory Tuning** Database-এর Memory Usage Optimize করতে সাহায্য করে।
- **Wait Events** Database কোথায় সময় ব্যয় করছে তা শনাক্ত করতে সাহায্য করে।
- **AWR Report** Oracle Database-এর Overall Workload ও Performance Analysis-এর জন্য ব্যবহৃত হয়।
- **ASH Report** Active Session এবং নির্দিষ্ট সময়ের Database Activity বিশ্লেষণে গুরুত্বপূর্ণ।

সবচেয়ে গুরুত্বপূর্ণ বিষয় হলো—

Performance Tuning মানে শুধু Index তৈরি করা নয়। প্রথমে Problem Identify করতে হবে, তারপর Evidence বা Performance Data বিশ্লেষণ করে Root Cause বের করতে হবে এবং শেষে উপযুক্ত Solution প্রয়োগ করে আবার Performance Measure করতে হবে।

এভাবেই একটি Database-কে Fast, Stable এবং Scalable রাখা সম্ভব।